



UDEM

UNIVERSIDAD DE MONTERREY

Práctica 7

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo GarcíaMat: 612348

Nombre: Ricardo Garza BenítezMat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 11 de junio del 2026

Introducción

Una de las capacidades que distingue a un microcontrolador como el PIC16F887 de un circuito puramente digital es su habilidad para interactuar con señales analógicas mediante un módulo conversor analógico-digital (ADC) integrado. Gracias a este módulo, el microcontrolador puede tomar un voltaje continuo, generado por ejemplo a partir de un sensor o de un divisor resistivo, y transformarlo en un valor numérico de 10 bits que puede ser procesado, almacenado o desplegado en pantalla. En esta práctica se explota precisamente esta capacidad para construir un pequeño sistema de medición con interfaz seleccionable.

La práctica consta de una sola actividad cuyo objetivo es leer de forma continua el voltaje analógico presente en el pin AN0, proveniente de un potenciómetro (RV2) que actúa como divisor de voltaje, y mostrar dicha lectura en una pantalla LCD bajo tres formatos distintos: el valor digital crudo entregado por el ADC (0 a 1023), su equivalente en voltaje (0 a 5 V) y su equivalente porcentual (0 % a 100 %). El cambio entre estos tres modos de visualización se controla mediante un botón conectado al pin RB0, configurado como interrupción externa (INT), de manera que cada pulsación incrementa un contador de modo que cicla entre las tres opciones.

Este ejercicio integra varios bloques fundamentales que se han venido trabajando a lo largo del curso: la configuración y lectura del módulo ADC, el manejo de una pantalla LCD en modo de 4 bits, y el uso de interrupciones externas para construir una interfaz de usuario sencilla pero funcional. Además, refuerza la importancia de mantener una correcta correspondencia entre la frecuencia del cristal instalado en la tarjeta y los parámetros de configuración del oscilador declarados en el código, aspecto que se retoma más adelante en las conclusiones.

Marco Teórico

Módulo Conversor Analógico-Digital (ADC) del PIC16F887

El PIC16F887 cuenta con un módulo ADC de 10 bits capaz de convertir una señal analógica de entrada en un valor digital comprendido entre 0 y 1023, con hasta catorce canales de entrada disponibles (AN0 a AN13). La configuración del módulo se realiza principalmente a través de los registros ADCON0 y ADCON1: el primero selecciona el canal de conversión mediante los bits CHS, habilita el módulo con el bit ADON, define el reloj de conversión con los bits ADCS y contiene el bit GO/DONE que inicia la conversión y se limpia automáticamente al terminar; el segundo controla la justificación del resultado mediante el bit ADFM y la referencia de voltaje mediante los bits VCFG. El resultado de la conversión se almacena en el par de registros ADRESH y ADRESL. En el código de esta práctica, ANSEL = 0x01 habilita únicamente el pin AN0 como entrada analógica, ADCON0 = 0x81 selecciona el canal AN0, activa el módulo y define el reloj de conversión, mientras que ADCON1 = 0x80 justifica el resultado a la derecha, de modo que ADRESH solo contiene los dos bits más significativos y ADRESL los ocho menos significativos [1].

El potenciómetro como divisor resistivo de voltaje

Para generar la señal analógica de entrada se utiliza un potenciómetro (RV2) conectado entre la alimentación de 5 V y tierra, con su terminal central (cursor) conectada al pin AN0. Este arreglo corresponde a un divisor de voltaje resistivo, en el cual el voltaje obtenido en el cursor depende de la posición relativa del mismo dentro de la resistencia total del potenciómetro, pudiendo variar de forma continua entre 0 V y el voltaje de alimentación. Al combinarse con un ADC de 10 bits y una referencia de 5 V, cada incremento unitario del valor digital corresponde aproximadamente a 4.89 mV, lo cual define la resolución mínima que el sistema puede distinguir. Esta configuración es muy común como banco de pruebas para sistemas de adquisición de datos, ya que permite simular de forma sencilla y repetible el comportamiento de un sensor analógico real [3].

Pantalla LCD HD44780 (LM016L) en modo de 4 bits

La pantalla LCD utilizada (LM016L) está basada en el controlador HD44780, un estándar de la industria para módulos LCD alfanuméricos de caracteres. Este controlador se comunica mediante las líneas RS (selección de registro, datos o instrucción), RW (lectura o escritura) y E (habilitación, que sincroniza la transferencia), además de un bus de datos que puede operar en modo de 8 o 4 bits. En esta práctica se emplea el modo de 4 bits, en el cual únicamente se utilizan las líneas D4 a D7 del controlador, transmitiendo cada byte en dos nibbles consecutivos; esto reduce el número de pines del microcontrolador necesarios para controlar la pantalla, a costa de duplicar el número de transferencias. La librería lcd.h utilizada encapsula esta complejidad mediante funciones como LCD_Init, LCD_Clear, LCD_Set_Cursor y LCD_puts, que internamente generan las secuencias de pulsos en RS, RW y E requeridas por el controlador [2].

Interrupción externa RB0/INT y el registro INTCON

El PIC16F887 permite que el pin RB0 funcione como una entrada de interrupción externa dedicada (INT). Su comportamiento se controla principalmente desde el registro INTCON, donde el bit GIE habilita globalmente las interrupciones, el bit INTE habilita específicamente la interrupción externa y el bit INTF es la bandera que se activa cuando ocurre el evento y debe limpiarse por software dentro de la rutina de servicio. Adicionalmente, el bit INTEDG del registro OPTION_REG determina si la interrupción se genera con un flanco de subida o de bajada en RB0. En el código de esta práctica se configuran GIE = 1, INTE = 1 e INTEDG = 0, de modo que la interrupción se dispara con un flanco de bajada, es decir, cuando el botón conectado a RB0 se presiona y lleva el pin a nivel bajo. Dentro de la rutina ISR, la función mode_change incluye además un pequeño retardo de 50 ms que actúa como mecanismo de antirrebote (debounce), evitando que un solo pulso mecánico del botón se interprete como varias pulsaciones consecutivas [1].

Configuración del oscilador: bits de configuración FOSC y la constante _XTAL_FREQ

La sección de configuración (#pragma config) del PIC16F887 incluye el campo FOSC, que determina el tipo de oscilador con el que operará el microcontrolador: opciones como LP, XT, HS o RC están pensadas para distintos rangos de frecuencia del cristal externo, siendo XT adecuado para cristales de baja a media frecuencia (hasta aproximadamente 4 MHz) y HS para cristales de alta frecuencia, como el de 16 MHz instalado en la tarjeta

de desarrollo utilizada en el laboratorio. De manera independiente, en el código se define la constante `_XTAL_FREQ`, la cual no configura ningún registro del hardware, sino que es utilizada exclusivamente por el compilador XC8 para calcular los ciclos de espera correctos dentro de las funciones `__delay_ms` y `__delay_us`. Por ello, una discrepancia entre la frecuencia real del cristal, el valor declarado en `FOSC` y el valor de `_XTAL_FREQ` no impide que el programa compile, pero sí altera la duración real de todos los retardos del programa, lo cual puede afectar el muestreo del ADC y los tiempos de refresco de la pantalla [1] [3].

Simulación

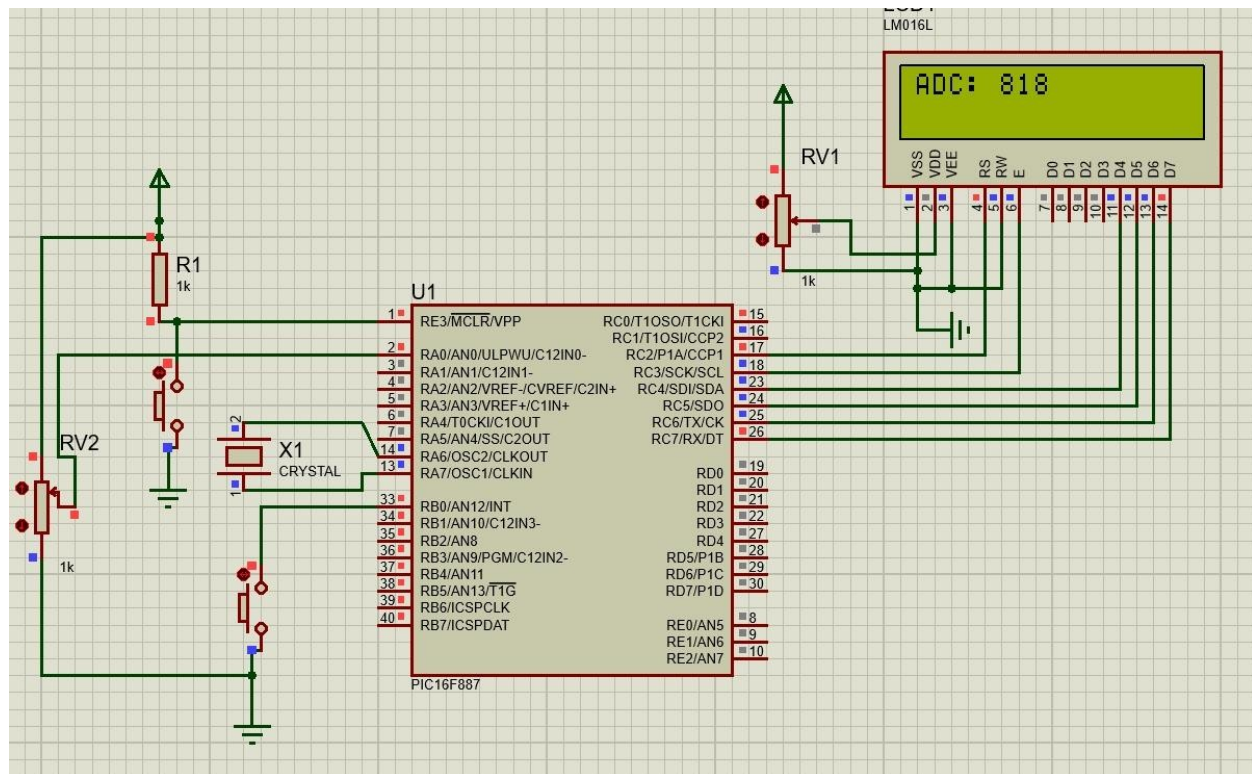
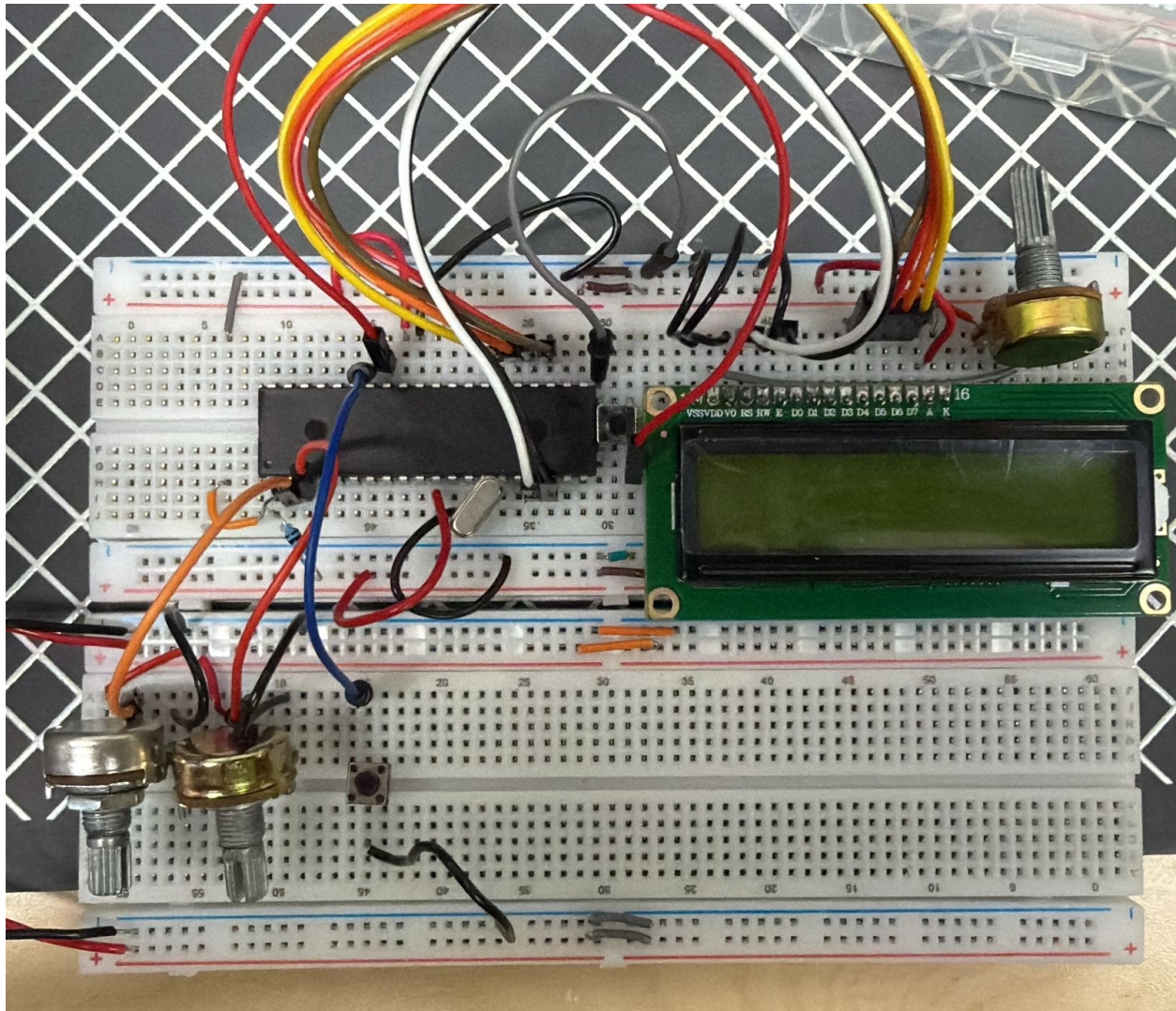


Foto del circuito en Físico



Resultados

Al energizar el circuito, la pantalla LCD inicia mostrando el modo por defecto (`disp_mod = 0`), correspondiente a la lectura de voltaje, en el formato "Voltaje: X.XXXX". Al girar el potenciómetro RV2 desde un extremo al otro, se observó que el valor mostrado variaba de forma continua y monótona entre aproximadamente 0.0000 V y 4.8900 V, lo cual es consistente con la resolución de 10 bits del ADC y con la fórmula de conversión empleada en el código ($\text{adc_result} \times 50000 / 1023$), que expresa el resultado con cuatro cifras decimales mediante la separación manual de la parte entera y la parte fraccionaria antes de formatearlas con `sprintf`.

Al presionar el botón conectado a RB0, la interrupción externa incrementó correctamente la variable `disp_mod`, ciclando la pantalla entre los tres modos de operación: "Voltaje: X.XXXX", "Porcentaje: XX %" y "ADC: XXX", regresando al primero después del tercero.

En el modo de porcentaje, los valores observados variaron de 0 % a 100 % de forma proporcional al giro del potenciómetro, mientras que en el modo de ADC puro se observaron valores enteros entre 0 y 1023, coincidiendo en todos los casos con la imagen de referencia capturada de la simulación, en la que se aprecia la lectura "ADC: 818" para una posición intermedia-alta del potenciómetro.

En general, el comportamiento del circuito físico fue consistente con el simulado en Proteus: las tres representaciones del mismo valor analógico (cuenta cruda, voltaje y porcentaje) se mantuvieron coherentes entre sí en todo momento, sin saltos erráticos ni lecturas espurias, y el cambio de modo mediante el botón resultó confiable y sin rebotes perceptibles gracias al retardo incluido en la rutina de interrupción. Esto confirma que tanto la configuración del módulo ADC como la inicialización y el manejo de la pantalla LCD en modo de 4 bits, así como la lógica de interrupción externa, funcionaron de acuerdo con lo esperado.

Conclusiones Individuales

Ricardo Garza

Para esta práctica me tocó armar físicamente la parte analógica del circuito, es decir, conectar el potenciómetro RV2 como divisor de voltaje hacia el pin AN0 y verificar con el multímetro que el rango de voltaje entregado realmente cubriera de 0 a 5 V conforme se gira la perilla; esto es importante porque si el potenciómetro está mal alimentado o el cursor no hace buen contacto, el ADC simplemente reporta un valor fijo o con saltos, y de entrada uno podría pensar que el error está en el código cuando en realidad está en el cableado. También me encargué de revisar la configuración del oscilador antes de programar: noté que el código declara `FOSC = XT` junto con `_XTAL_FREQ` de 8 MHz, mientras que el cristal que tenemos instalado en la tarjeta es de 16 MHz, así que antes de cargar el programa platicamos sobre si convenía ajustar estos valores para que coincidieran con el cristal real, ya que de las prácticas anteriores aprendimos que un descuido así no impide compilar, pero sí desfasa todos los retardos del programa. Al final, con la frecuencia que dejamos, las lecturas en el LCD se mantuvieron estables y consistentes con la simulación, pero quedo con la idea, de cara a las siguientes prácticas, de hacer esta verificación de manera sistemática desde el principio y no como un paso de último momento.

Eugenio Romo

En esta práctica me quedó más claro cómo el PIC16F887 traduce una señal analógica continua en un número manejable por software: el ADC entrega un valor entre 0 y 1023, y a partir de ese único dato el programa puede generar tres representaciones distintas (cuenta cruda, voltaje y porcentaje) simplemente aplicando operaciones aritméticas diferentes sobre el mismo resultado. También entendí mejor la diferencia entre configurar un periférico mediante registros, como `ADCON0` y `ADCON1` para el ADC o `INTCON` y `OPTION_REG` para la interrupción de RB0, y comunicarse con un dispositivo externo como la pantalla LCD a través de una librería que ya encapsula el protocolo del controlador HD44780. Ver cómo una sola interrupción externa puede controlar de forma limpia el cambio entre varios modos de visualización me parece un patrón muy útil que

seguramente volveremos a usar en proyectos donde se necesiten varias pantallas o menús dentro de un mismo programa.

Referencias

- [1] Microchip Technology. (2009). PIC16F882/883/884/886/887 Data Sheet. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>
- [2] Hitachi. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver. <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>
- [3] Merino Pérez, L. A. (2012). Microcontroladores PIC: diseño práctico de aplicaciones (3.^a ed.). McGraw-Hill.